202301978 컴퓨터융합학부 DS_01 week5 assignment

```java
public static class Node <E>  7개 사용 위치  doyun
{
    private E item;  3개 사용 위치
    private Node<E> next;  3개 사용 위치
    public Node(E newItem, Node<E> node)  1개 사용 위치  doyun
    {
        item = newItem;
        next = node;
    }

    public E getItem() {return item;}  2개 사용 위치  doyun
    public Node<E> getNext() {return next;}  1개 사용 위치  doyun
    public void setItem(E newItem) {item = newItem;}  0개의 사용위치  doyun
    public void setNext(Node<E> newNext) {next = newNext;}  0개의 사용위치  doyun
}
```

Singly Linked List를 활용해 스택을 구현할 것이기에 먼저 Node를 정의해준다. Item과 다음 노드의 주소를 가질 next를 만들고 method를 통해 해당 값에 접근하고 수정할 수 있도록 하였다.

```java
public static class ListStack <E>  4개 사용 위치  doyun
{
    private Node<E> top;  7개 사용 위치
    private int size;  5개 사용 위치
    public ListStack()  2개 사용 위치  doyun
    {
        top = null;
        size = 0;
    }
    public int size() {return size;}  2개 사용 위치  doyun
    public boolean isEmpty() {return size == 0;}  9개 사용 위치  doyun

    public void push(E newItem)  20개 사용 위치  doyun
    {
        Node<E> newNode = new Node<>(newItem, top);
        top = newNode;
        size++;
    }
    public E pop()  20개 사용 위치  doyun
    {
        if(isEmpty()) {
            throw new EmptyStackException();
        }
            E returnItem = top.getItem();
            top = top.getNext();
            size--;
            return returnItem;

    }
```

```java
    public E peek()  7개 사용 위치  doyun
    {
        if(isEmpty())
        {
            throw new EmptyStackException();
        }
        return top.getItem();
    }
}
```

ListStack class를 선언하고 내부에 size, isEmpty, push, pop, peek함수를 구현한다.

```java
public static int operation(int a, int b, String operator)  6개 사용 위치  doyun *
{
    if(operator.equals("*")) return a * b;

    else if(operator.equals("/")) return a / b;

    else if(operator.equals("-")) return a - b;

    else return a + b;
}
```

```java
public static int compareOperator(String op1, String op2)  6개 사용 위치  doyun *
{
    int op1Num = 0;
    int op2Num = 0;

    if(op1.equals("*") || op1.equals("/")) op1Num = 2;

    else if(op1.equals("+") || op1.equals("-")) op1Num = 1;

    if(op2.equals("*") || op2.equals("/")) op2Num = 2;

    else if(op2.equals("+") || op2.equals("-")) op2Num = 1;

    return op1Num - op2Num;
}
```

많이 중복될 요소인 계산과 연산자 우선순위 비교 알고리즘은 따로 함수로 만들어 코드가 너무 길어지는 것을 방지 했다.

Operation 함수 같은 경우 각 연산자에 맞는 연산을 return 해주고 compareOperator함수의 경우 연산자의 우선순위를 비교해 *, /의 경우 2, +, -의 경우 1, (,) 의 경우 0이 되도록 한 후 op1 – op2 값을 반환한다. Op1이 op2보다 우선순위가 더 높을 경우 0 보다 크고 Op1이 op2보다 우선순위가 더 낮을 경우 0보다 작다는 점을 통해 어떤 연산자의 우선순위가 더 높은지 확인할 수 있다.

```java
public static int calc_infix(String s)  1개 사용 위치  doyun
{
    ListStack<Integer> operandList = new ListStack<Integer>();
    ListStack<String> operatorList = new ListStack<String>();
    StringTokenizer st = new StringTokenizer(s);
    int size = st.countTokens();

    for(int i = 0; i < size; i++)
    {
        String input = st.nextToken();
        int p;

        if(input.equals("(")) operatorList.push( newItem: "(");

        else if(input.equals(")"))
        {
            while(!operatorList.peek().equals("("))
            {
                int b = operandList.pop();
                int a = operandList.pop();
                p = operation(a, b, operatorList.pop());
                operandList.push(p);
            }
            operatorList.pop();
        }
else if(input.equals("/"))
{
    if(!operatorList.isEmpty())
    {
        if(compareOperator(input, operatorList.peek()) > 0)
        {
            operatorList.push( newItem: "/");
        }
        else
        {
            int b = operandList.pop();
            int a = operandList.pop();
            p = operation(a, b, operatorList.pop());
            operandList.push(p);
            operatorList.push( newItem: "/");
        }
    }
    else
    {
        operatorList.push( newItem: "/");
    }
}
else if(input.equals("+"))
{
    if(!operatorList.isEmpty())
    {
        if(compareOperator(input, operatorList.peek()) > 0)
        {
            operatorList.push( newItem: "+");
        }
        else {
            while (compareOperator(input, operatorList.peek()) <= 0 && operandList.size() >= 2) {
                int b = operandList.pop();
                int a = operandList.pop();
                p = operation(a, b, operatorList.pop());
                operandList.push(p);

                if (operatorList.isEmpty()) {
                    break;
                }
            }
            operatorList.push( newItem: "+");
        }
    }
    else
    {
        operatorList.push( newItem: "+");
    }
}
```

```java
else if(input.equals("*"))
{
    if(!operatorList.isEmpty())
    {
        if(compareOperator(input, operatorList.peek()) > 0)
        {
            operatorList.push( newItem: "*");
        }
        else
        {
            int b = operandList.pop();
            int a = operandList.pop();
            p = operation(a, b, operatorList.pop());
            operandList.push(p);
            operatorList.push( newItem: "*");
        }
    }
    else
    {
        operatorList.push( newItem: "*");
    }
}
```

```java
else if(input.equals("-"))
{
    if(!operatorList.isEmpty())
    {
        if(compareOperator(input, operatorList.peek()) > 0)
        {
            operatorList.push( newItem: "-");
        }
        else
        {
            while (compareOperator(input, operatorList.peek()) <= 0 && operandList.size() >= 2) {
                int b = operandList.pop();
                int a = operandList.pop();
                p = operation(a, b, operatorList.pop());
                operandList.push(p);
                if (operatorList.isEmpty()) {
                    break;
                }
            }
            operatorList.push( newItem: "-");
        }
    }
    else
    {
        operatorList.push( newItem: "-");
    }
}
else
{
    operandList.push(Integer.parseInt(input));
}

    while(!operatorList.isEmpty())
    {
        int b = operandList.pop();
        int a = operandList.pop();
        int p = operation(a, b, operatorList.pop());
        operandList.push(p);
    }
    return operandList.pop();
}
```

Calc_infix 함수에서는 들어온 input String을 글자의 종류에 따라 다르게 처리한다.

숫자, 즉 피연산자의 경우 operandList stack에 push하고

"(", ")", 괄호의 경우 "("는 괄호안의 내용이 어디까지 인지 구분 지어야 하기에 operatorList stack에 push하고 ")"가 입력될 때 "("전 까지의 내용을 연산 후 stack에 삽입해준다. Stack의 가장 위에 "("가 있을 때까지 연산을 진행하며 마지막으로 "("을 꺼내주어 괄호 안 연산을 진행한다.

"*", "/", 우선순위가 높은 연산자인 경우 operatorList에 다른 연산자가 있는지 먼저 확인한다. 비어 있다면 "*", "/"를 push한다. 다른 연산자가 있다면 stack 제일 위 연산자와 비교를 먼저 진행한다. "*", "/"의 우선 순위가 더 크다면 push를 진행하고 작거나 같다면 operatorList.peek() 연산자로 연산을 진행한 후 연산 결과 내용을 operandList에 push하고 그 다음 "*", "/"를 operatorList에 push한다.

"+", "-" 우선순위가 낮은 연산자의 경우 역시 operatorList에 다른 연산자가 있는지 먼저 확인한다. operatorList가 비어 있다면 "+", "-"를 push한다. 다른 연산자가 있다면 stack 제일 위 연산자와 비교를 진행한다. "+", "-"보다 우선순위가 낮은 연산자는 "("말고는 없기에 "("가 stack의 가장 위 요소인 경우 "+","-"를 push하고 아니라면 피연산자가 2개가 남고, "+","-" 보다 우선순위가 높은 연산자가 stack안에 있을 때 계속 연산을 진행한다.

"-"와 "/"의 경우 A-B, A/B와 B-A, B/A의 결과가 다르기에 operation 함수에 피연산자를 매개변수로 넣을 때 순서에 주의해야 한다.

Input String의 글자 개수만큼 st.token이 피연산자인지, 연산자인지 구분 후 각 케이스별로 연산 및 push를 진행한다. 그 후에는 각 stack에 남아있는 요소들을 처리하기 위해 operatorList stack이 비워질때 까지 연산을 진행한 후 operandList stack에 남아있는 마지막 요소를 반환하여 최종 연산 결과를 반환하도록 한다.

```java
public static void main(String[] args) throws IOException  doyun
{
    try
    {
        File file = new File( pathname: "스택/test.txt");
        FileReader fr = new FileReader(file);
        BufferedReader br = new BufferedReader(fr);
        String line = br.readLine();

        while(line != null)
        {
            System.out.println(calc_infix(line));
            line = br.readLine();
        }
        br.close();
    }
    catch(FileNotFoundException e)
    {
        System.out.println("file is not found");
    }
    catch(IOException e)
    {
        System.out.println("I/O exception");
    }

}
}
```

Text.txt에서의 한줄을 그대로 calc_infix함수에 넣어주고 반환값을 출력하도록 한다.

```
operation : 4
operation : 20
operation : 23
operation : 14
14
operation : 4
operation : 20
operation : 5
5
operation : 3
operation : 2
operation : 8
operation : 11
11
```

Operand 함수가 실행될 때마다 연산 결과를 출력해보면 다음과 같다.